P2664R2
Proposal to extend std::simd with permutation API

Published Proposal, 2023-04-28

This version:
http://wg21.link/P2664R2.pdf

Authors:
Daniel Towner (Intel)
Ruslan Arutyunyan (Intel)

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Proposal to extend `std::simd` with features enabling permutation of data within and across SIMD values.

Table of Contents

§ 1. Motivation

ISO/IEC 19570:2018 introduced data-parallel types to the C++ Extensions for Parallelism TS. [P1915R0] asked for feedback on `std::simd` in Parallelism TS 2. [P1928R3] aims to make that a part of C++ IS. Intel supports the concept of a standard interface to SIMD instruction capabilities and has made further suggestions for other APIs and facilities for `std::simd` in document [P2638R0].

One of the extra features we suggested was the ability to be able to permute elements across or within SIMD values. These operations are critical for supporting formatting of data in preparation for other operations (e.g., transpose, strided operations, interleaving, and many more), but are relatively poorly supported in the current `std::simd` proposal. Many modern SIMD processors include sophisticated support for permutation instructions so in this document we shall propose a set of API functions which make those underlying instructions accessible in a generic way.

In this document we shall start with some background into the current `std::simd` proposal and extensions, describe the state of the art on other simd libraries, and then describe a proposal for some new API functions to add to `std::simd`.

§ 2. Background

In ISO/IEC 19570:2018 there were only a few functions which could be used to achieve any sort of element permutation across or within `simd<>` values. Those functions were relatively modest in scope and they built upon the ability to split and concatenate simd values at compile-time. For example, a simd value containing 5 elements could be split into two smaller pieces of sizes 2 and 3 respectively:

```
fixed_size_simd<float, 5> x;
…
const auto [piece0, piece1] = split<2, 3>(x);
```

Those smaller pieces could be acted on using any of the `std::simd` features, and then later recombined using `concat`, possibly in a different order or with duplication:

```
const auto output = concat(piece1, piece1, piece0);
```

The `split` and `concat` functions can be used to achieve several permutation effects, but they are unwieldy for anything complicated, and since they are also compile-time they preclude any dynamic runtime permutation.

In [P2638R0] Intel suggested extra operations to insert and extract SIMD values to and from other SIMD containers:

```
const auto t = extract<2, 5>(v); // Read out 3 SIMD values, starting from p
insert<5>(v, t); // Put the 3 previously read values back into the containe
```

These are convenient, but they still don't allow arbitrary or expressive permute operations.

Two suggestions were made in [P0918R1]: add an arbitrary compile-time shuffle function, and add a specific type of permutation called an interleave. The compile-time shuffle can be used as follows:

```
const auto p = shuffle<3, 4, 1, 1, 2, 3>(x);
// p::value_type will be the same as x::value_type
// p::size will be 6
```

The indexes can be arbitrary, and therefore allow any duplication and reordering of elements. The output `std::simd` object will have the same number of elements as there are supplied indexes, and the type of the SIMD output will be that of the input. It is noted also in the shuffle API that this

function can be applied to `simd_mask` values too, and that shuffling across multiple SIMD values can be achieved by first concatenating the values into a single larger SIMD:

```
const auto p = shuffle<10, 9, 0, 1>(concat(x, y));
```

In addition to the shuffle function, [P0918R1] also proposes a function called interleave which accepts two SIMD values and interleaves respective elements into a new SIMD value which is twice the size. For example, given inputs `[a0 a1 a2]` and `[b0 b1 b2]`, the output would be `[a0 b0 a1 b1 a2 b2]`. Interleaving is a common operation and is often directly supported by processors with explicit instructions (e.g., Intel's `punpcklwd`), so as in [P0918R1] and also in other simd libraries there will be named functions which expose that instruction. There will be other common permutation operations which also have specialist hardware support and it is common for them to be exposed in named functions also. For example, [Highway] provides `DupOdd`, `DupEven`, `ReverseBlocks`, `Shuffle0231`, and so on, which map efficiently to underlying instructions. While this can ensure that good code is generated for specific function, it does mean that:

- the set of named functions can only include those features available on all potential targets
- portability is reduce by exposing functions only on those targets which support them.

Neither of these is desirable, and in our suggestions below we provide an alternative.

§ 3. Extending `std::simd` to support permutations

There are three classes of permutation which could be supported by `std::simd`;

- using compile-time computed indexes
- using another SIMD as a dynamic index
- using a bit-mask for selecting active elements

Modern processor instruction sets typically support all 3 of these classes of instruction. In the following sections we shall examine in more detail each of these classes and the potential APIs needed to expose those features to the user. We shall also examine what it means to permute memory, and whether named functions should be provided for common permutation patterns.

`std::simd` could be modified to allow permutations either by adding to the base definition of `std::simd` itself (specifically, to include a new `simd::operator[]` or `simd_mask::operator[]`), or by introducing a new overloaded free-functions which extend `std::simd`. We shall consider both options below.

Note that a `simd_mask` is a special case of a type of SIMD and can therefore be permuted in the same way as a conventional SIMD. In our discussions below we assume that the permutations would apply equally to a `simd_mask` unless we note otherwise.

§ 3.1. Using compile-time computed indexes

The first proposal is to provide a `permute` function which accepts a compile-time function which is used to generate the indexes to use in the permutation. This is a very powerful concept: firstly, it allows the task of computing the indexes to be offloaded to the compiler using an index-generating function, and secondly it works on `simd<>` values of arbitrary size. It's definition would be:

```
template<std::size_t OutputSize = 0, typename T, typename Abi>
constexpr simd<T, simd_abi::deduce_t<T, new_simd_size, Abi>>
permute(const simd<T, Abi>& v, std::invocable<std::integral> auto fn)
```

It can be used as in the following example:

```
const auto dupEvenIndex = [](size_t idx) -> size_t { return (idx / 2) * 2;
const auto de = permute(values, dupEvenIndex);
```

Note that the permutation function maps from the index of each output element to the index which should be used as the source for that position. So, for example, the `dupEvenIndex` function would map the output indexes `[0, 1, 2...)` to the source indexes `[0 0 2 2 4 4…)`. This example permutation is common enough that it has hardware support in Intel processors, and the compiler can map the above function directly to the corresponding instruction:

```
vmovsldup ymm0, ymm0
```

By default, the permute function will generate as many elements in the output simd as there were input values, but the output size can be explicitly specified if it cannot be inferred or it needs to be modified. For example, the following permute generates 4 values with a stride of 3 (i.e., indexes `[0, 3, 6, 9]`).

```
const auto stride3 = permute<4>(values, [](size_t idx) -> size_t { return
idx * 3; });
```

3.1.1. Design option - extended indexes

The obvious index representation to return from each generator invocation is to return an integral value in the range `[0...size)`. The compiler can enforce at compile-time that the index is in the valid range and dynamic indexes would not be permitted. However, there are three ways in which the set of permitted indexes could be extended to give more flexibility, and to tell the compiler more about the intent of the programmer for a given permutation.

Firstly, the indexes could be extended to allow negative indexes, which would be interpreted as indexes taken from the end of the simd instead of the beginning. So -1 would be the last element, -2 the penultimate element, and so on. This feature can be found in other languages.

Secondly, the indexes could allow the special value 0 to be inserted. This would need to be a special constant which is guaranteed not to clash with a valid index. Inserting a zero could also be achieved by concatenating a zero-initialized simd and selecting elements from that instead:

```
permute(concat(v, simd<T>()), [](auto idx) { (idx % 2) ? v.size : idx; });
```

which with a zero constant could instead be written as:

```
permute(v, [](auto idx) { (idx % 2) ? simd_zero_element : idx; });
```

Thirdly, it can be convenient and efficient to hint to the compiler that some elements of the simd need not be given values. There could be special constant called `simd_uninit_element` which indicates to the compiler that it can use an instruction sequence which is permitted to leave some values uninitialized if that is faster.

3.1.2. Design option - pass in size

It may be convenient to allow the permutation callable function to accept an optional parameter giving the size of the incoming SIMD, in addition to the index to generate. For example, a reverse function could be:

```
auto reverser = (size_t index, size_t size) { return size - index; }
```

There is likely no need for this if extended indexes can be used to directly address elements from the end.

§ 3.1.3. Implementation Experience

In other simd or vector libraries which provide named specific permute functions, the reason often given is that it allows specific hardware instructions to be used to efficiently implement those permutes. For example, Intel has the `vmovsldup` instruction as noted above, so some libraries provide functions in their API with names which reflect this (e.g., such a function might be called `DupLow`). The disadvantages are that it hinders portability, and only allows access to the hardware features that the authors of the library have chosen to expose.

Using the generator-based approach makes it easier to access a wide range of underlying special purpose instructions, or to fall back to equivalent instructions sequences where they are not available, but it is desirable that such behavior should be efficient. To judge whether the generator-based permutation API was efficient and useful we implemented the extensions in the Intel `std::simd` library and looked at the performance of different compile-time patterns and their generated instruction sequences. We found that GCC and LLVM were able to determine the correct hardware instructions to use for a variety of common permutation patterns, and in some cases the compiler was even able to use the side-effects of other instructions to effect permutations in ways that human programmers had overlooked. In all cases, where the pattern was too complicated to map to native hardware features the compiler fell back to using general purpose permutation patterns, such as Intel's `permutex2var` family of instructions.

As a small illustration of the compiler's ability, the following table shows some compile-time function permute calls and the code that the LLVM 14.0.0 compiler has generated for each:

Permute call	Purpose	Output from clang 14.0.0
`permute (x, [](auto idx) { return idx & ~1; });`	Duplicate even elements	`vmovsldup zmm0, zmm0`
`permute (x, [](auto idx) { return idx ^ 1; });`	Swap even/odd elements in each pair (complex-valued IQ swap)	`vpermilps ymm0, ymm0, 177`
`permute<8>(x, [] (auto idx) { return idx + 8; });`	Extract upper half of a 16-element vector. Note that the instruction sequence accepts a zmm input and returns a ymm output.	`vextractf64x4 ymm0, zmm0, 1`

In each case the compiler has accepted the compile-time index constants and converted them into an instruction which efficiently implements the desired permutation. We can safely leave the compiler to determine the best instruction from an arbitrary compile-time function.

§ 3.2. Using another SIMD as the dynamic index

The second proposal for the permute API is to allow the required indexes to be passed in using a second SIMD value containing the run-time indexes:

```
template<typename T, typename AbiT, std::integral U, typename AbiU>
constexpr resize_simd_t<simd_size_v<U, AbiU>, simd<T, AbiT>>
permute(const simd<T, AbiT>& v, const simd<U, AbiU>& indexes)
```

This can be used as in the following example:

```
fixed_size_simd<unsigned, 8> indexes = getIndexes();
fixed_size_simd<float, 5> values = getValues();
...
const auto result = permute(values, indexes);
// result::value_type is float
// result::size is 8
```

The permute function will return a new SIMD value which has the same element type as the input value being permuted, and the same number of elements as the indexing SIMD (i.e., float and 8 in the example above). The permute may duplicate or arbitrarily reorder the values. The index values must be of integral type, with no implicit conversions from other types permitted. The behavior is undefined if any index is outside the valid index range. Dynamic permutes across multiple input values are handled by concatenating the input values together. The indexes in the selector will only be treated as indexes, and will never have special properties as described above (e.g., no zero element selection)

In addition to (*or instead of?*) the function called permute, a subscript operator will also be provided:

```
template <std::integral U, typename _AbiU>
constexpr resize_simd_t<simd_size_v<U, AbiU>, simd>
simd::operator[](const simd<U, AbiU> &indexes) const;
```

Here is an example of how this could be used:

```
fixed_size_simd<int, 8> indexes = getIndexes();
fixed_size_simd<float, 5> values = getValues();
...
```

```
const auto result = values[indexes];
// result::value_type is float
// result::size is 8
```

§ 3.2.1. Design option - C++23 multi-index subscript

Sometimes the subscripts to use in a permute might be known at compile time, but to use them in a permutation requires them to be put into an index simd first:

```
constexpr simd<int> indexes = {3, 6, 1, 6};
auto output = values[indexes];
```

In C++23 multi-subscript operator was introduced and this could be used to allow a more compact representation:

```
auto output = values[3, 6, 1, 6];
// output::value_type is values::value_type
// output::size is 4
```

It would be good if it could also appear on the left-hand-side;

```
output_simd[2, 5, 6] = x; // Overwrite selected elements with a single valu
```

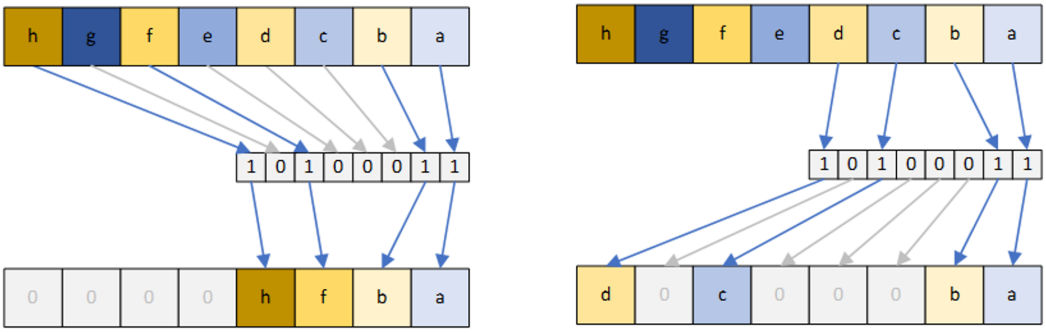
Should non-constant indexes be allowed too? Our experience with GCC and LLVM suggests that work would be needed on their code generation for non-constant variable indexes to be improved though, as they are both currently poor at this.

§ 3.2.2. Implementation experience

We implemented the runtime-indexed permute API in Intel's example `std::simd` implementation. Small index operations (e.g., indexing one native sized simd by another) were mapped directly onto the underlying native instruction and so were as efficient as calling an intrinsic. More complicated cases for permutation, such as where either the index or data SIMD parameters were bigger than a native register are also handled with comparable efficiency to hand-written intrinsic code.

§ 3.3. Permutation using simd masks (compress/expand)

A third variant of permutation is where a `simd_mask` is used as a proxy for an index sequence instead. The presence or absence of an element in the `simd_mask` will cause the element to be used or not. The following diagrams illustrate this:



On the left the values in the SIMD are compressed; only those elements which have their respective `simd_mask` element set will be copied into the next available space in the output SIMD. Their relative order will be maintained, and the selected elements will be stored contiguously in the output. The output value will have the same number of elements as the input value, with any unused elements being left in a valid but unspecified state. The behavior of the function is similar to `std::copy_if`, although other simd libraries may call this function compression or compaction instead. The expansion function on the right of the diagram has the opposite effect, copying values from a contiguous region to potentially non-contiguous positions indicated by a mask bit. The two functions have prototypes as follows:

```
template<typename T, typename Abi>
simd<T, Abi> compress(const simd<T, Abi>& value, const simd_mask<T, Abi>& m

template<typename T, typename Abi>
simd<T, Abi> expand(const simd<T, Abi>& value, const simd_mask<T, Abi>& mas
```

In addition to these named functions the subscript operator can also accept a mask:

```
auto compressedValues = simdValues[mask];
```

In all functions the output of the operation will have an element type which matches the simd and simd_mask element types being permuted, and has the size of the mask. The mask cannot be bigger than the simd value being permuted (to do so would mean that some masked elements are beyond the

end of the input values), but the mask can be smaller than the input value (the input value is effectively truncated).

3.3.1. Design option - lvalue reference

Should `operator[simd_mask]` be permitted on the left-hand side of an assignment so that selected output elements of a simd can be written?

```
simd_value[mask] = inputValues;
```

This is equivalent to doing an expansion where those values of `simd_value` for which the respective mask bit is set will be overwritten, and all other elements left as they were.

With this syntax it might be desirable to allow this too:

`simd_value[mask] = scalar;`

But that is no longer really a permute.

3.3.2. Design option - unused value element initialisation

Consider a compression operation:

```
fixed_size_simd<int, 10> values = ...;
fixed_size_simd_mask<int, 6> mask = ...;

auto compressed = compress(values, mask);
// compressed::value_type is int
// compressed::size is 6
```

The output value has the same size as the mask selector, but the actual mask bits won't be known until run-time. This raises the question of what values should be put into the unused output elements.

The current proposal would leave the unused elements in a valid but unspecified state. This is comparable to functions like `std::shift_left` and `std::shift_right`. The alternative is for the compress function to leave the unused values in a value-initialized state.

The advantage of an unspecified state is that the code doesn't need to make a special effort to insert values which might not be used anyway, but has the disadvantage that the unspecified state might

catch out the unwary user. Using a value initialized state helps the unwary user by providing a default state which is sane and repeatable, but may come with a performance cost.

Intel have an example implementation of `std::simd`, and our experience with that demonstrates that when native support is available (e.g., with AVX-512 ISA) it makes no difference to performance whether the unused values are initialising to specific value or left uninitialized, since the instruction itself inserts the values into unused positions as an integrated part of its operation. However, a synthesized code sequence is more expensive when setting the elements to a specific value since it is necessary to also compute the population count of the mask, turn that into a mask and then arrange for the unused values to be overwritten with something else using that mask.

Given that the compress function is essentially only extracting valid values from the input SIMD and discarding the others, and that there is a potential performance penalty that comes from initialising unused values, then we think that the default should be to leave unused elements in unspecified states.

Related to this is what to do when a programmer does want to initialize unused values to a known value. With the proposed interface the programmer would be required to compute a new partial mask from their original selection mask (i.e., compute the population count of the original mask, and then turn that into a mask of the lower elements), and then use that to conditionally blend in their desired value. This can be inefficient, and for targets like Intel AVX-512, it doesn't exploit the hardware's ability to automatically insert a value anyway. For this reason we propose that an overload of `compress` is provided that accepts a third parameter giving a value to insert into unused elements:

```
template<typename T, typename Abi>
simd<T, Abi> compress(const simd<T, Abi>& value, const simd_mask<T, Abi>& m
```

This makes the programmer's intent explicit in the code, it simplifies the code, and it allows those targets which support automatic insertion of a value to efficiently make use of that feature.

3.3.3. Design option - use permute as a name instead of compress

Rather than `compress`, should it be called `permute(value, mask)`? That is uniform compared to the other permutation functions. This would also require replacing `expand` with a call to `permute` where it appears as the destination:

```
output = permute(input, mask); // Compression - choose active elements
permute(output, mask) = input; // Expansion - write from contiguous input t
```

These look a bit odd in use, so `operator[]` is maybe more obvious.

```
output = input[simd_mask];
output[simd_mask] = input;
```

3.3.4. Implementation Experience

In Intel's example implementation of `std::simd` we have implemented permute-by-mask. On modern processors which support AVX-512 or VBMI2 we found the mapping from `compress` or `operator[]` to be efficient, and allowed a natural representation of this compaction operation where needed. Implementing permute-by-mask on older processors which lacked native support was a little harder, but could still be implemented as efficiently as an experienced programmer could achieve using hand-written intrinsics. Determining the most efficient implementation under all conditions however showed that less experienced programmers would struggle to implement this feature. Therefore, making this feature available in `std::simd` itself removed any non-portable calls to native compression/expansion instructions, and also allowed the library implementer to use efficient sequences for different scenarios.

3.4. Memory-based permutes

When permuting values which are stored in memory, the operations are normally called gather (reading data from memory), or scatter (writing values to memory). The existing proposals for permute above could be overloaded to also accept a memory pointer:

```
int array[1024];
...
const auto r0 = permute<6>(array, dupEven);
const auto r1 = permute(array, indexes);
```

Alternatively, these functions could be called gather and scatter to make it explicitly clear that they are operating on memory.

```
int array[1024];
...
const auto r0 = gather<6>(array, dupEven);
const auto r1 = gather(array, indexes);
```

Or if [DNMSO] is allowed, then a pointer or `std::contiguous_iterator` would be used directly:

```
m = v.begin();
auto data = m[simd_values];

m[simd_values] = inputs; // Scatter to memory
```

Note that the permute-by-mask operations could also be permitted on memory:

```
auto data = m[simd_mask]; // Analagous to std::copy_if
m[simd_mask] = inputs;    // Expand to memory
```

§ 4. Named functions

The APIs discussed in this document are very flexible and can be used to build effectively arbitrary permutations. However, there are inevitably certain patterns that are very common and it could be convenient to provide named functions for those patterns. For example:

```
auto dupEven(simdable v) {
    return permute(v, [](size_t idx) -> size_t { return (idx / 2) * 2; });
};
auto dupOdd(simdable v) {
    return permute(v, [](size_t idx) -> size_t { return idx | 1; });
};
auto swapOddEven(simdable auto v) {
    return permute(v, [](size_t idx) -> size_t { return idx ^1; });
}
auto even(simdable auto v) {
    return permute<v::size() / 2>(v, [](size_t idx) -> size_t { return idx
}
```

In many cases there are already names in C++ that reflect the operation. Providing overloads in `std::simd` which create SIMD functions with similar names to their existing C++ counterparts

makes it clear to the programmer what the function is doing when applied to a SIMD value. For example:

```
std::rotate
std::shift_left
std::shift_right
std::copy_if
std::remove_if
std::slice // tricky - used for valarray at the moment, but it
           // also captures concepts like odd and even
std::stable_partition // Use a mask to split the values
```

These functions could be added at a later date.

5. Wording

TBD

6. Summary

In this document we have described three styles of interface for handling permutes: compile-time generated, simd-indexed, and mask-indexed. In combination, these three interfaces allow all types of common permute to be expressed in a way which is clean, concise, and efficient for the compiler to implement.

References

Informative References

[DNMSO]
Matthias Kretz. *Non-Member Subscript Operator*. URL: https://web-docs.gsi.de/~mkretz/DNMSO.pdf

[Highway]
Google. *Google Highway*. URL: https://github.com/google/highway/

[P0918R1]
Tim Shen. *More simd<> Operations*. 29 March 2018. URL: https://wg21.link/p0918r1

[P1915R0]

Matthias Kretz. *Expected Feedback from simd in the Parallelism TS 2*. 7 October 2019. URL: https://wg21.link/p1915r0

[P1928R3]

Matthias Kretz. *Merge data-parallel types from the Parallelism TS 2*. 3 February 2023. URL: https://wg21.link/p1928r3

[P2638R0]

Daniel Towner. *Intel's response to P1915R0 for std::simd parallelism in TS 2*. 22 September 2022. URL: https://wg21.link/p2638r0